Phase 2 – Vom Modell zur sauberen Architektur

Einführung in objektorientierte Programmierung mit Python

1 Grundidee: Warum ein Schichtenmodell?

Beim Einstieg in größere Programme, etwa das Dashboard zur Studienorganisation, entsteht schnell der Wunsch, alle Aufgaben in einer Klasse oder Datei zu bündeln. Solange Programme sehr klein sind, scheint dieser Ansatz praktikabel. Mit jeder Erweiterung nimmt jedoch die Komplexität zu, und Änderungen an einer Stelle haben unerwartete Auswirkungen an anderen Stellen. Ein Schichtenmodell begegnet diesem Problem, indem Verantwortlichkeiten klar getrennt werden. Jede Klasse übernimmt eine klar abgegrenzte Aufgabe. Änderungen an der Darstellung sollen keine fachliche Logik beeinflussen, und ein Wechsel der Datenquelle soll nicht zu umfangreichen Anpassungen im gesamten Code führen. Eine zentrale Leitidee dabei ist das Single Responsibility Principle aus den SOLID Regeln. Jede Klasse soll genau einen klaren Grund zur Änderung haben.

Teil I

Minimalanforderungen an eine saubere Architektur

Dieser erste Teil beschreibt die minimale Struktur, die erforderlich ist, um ein Dashboard übersichtlich und erweiterbar umzusetzen. Ziel ist es, von Beginn an saubere Zuständigkeiten einzuführen, ohne Anfänger:innen mit Details zu überfordern.

2 Domain Klassen

Domain Klassen beschreiben die fachlichen Objekte des Systems. Sie waren Hauptbestandteil von Phase 1. Im Kontext eines Studien Dashboards sind dies beispielsweise Module und Prüfungsleistungen.

Datei: `pruefungsleistung.py`

```
1 from dataclasses import dataclass
2
```

```python
@dataclass
class Pruefungsleistung:
    """Domain-Klasse: z.B. Klausur, Hausarbeit, Projekt."""

    titel: str
    punkte: int | None = None  # None = noch nicht bewertet

    def ist_bewertet(self) -> bool:
        return self.punkte is not None
```

Datei: `modul.py`

```python
from __future__ import annotations
from dataclasses import dataclass, field

@dataclass
class Modul:
    """Domain-Klasse: repräsentiert ein Studienmodul."""

    name: str
    ects: int
    pruefungsleistungen: list["Pruefungsleistung"] = field(default_factory=list)

    def fuege_pruefungsleistung_hinzu(self, pruefung: "Pruefungsleistung") ->
    ↪ None:
        self.pruefungsleistungen.append(pruefung)
```

Domain Klassen können mithilfe von `@dataclass` umgesetzt werden. Eine Dataclass erzeugt automatisch Methoden wie den Konstruktor und reduziert damit technischen Ballast („Boilerplate-Code“). Dies verbessert die Lesbarkeit und unterstützt Anfänger:innen beim Verständnis der Klassenstruktur.

Die Anweisung `from __future__ import annotations` sorgt dafür, dass Typannotationen später ausgewertet werden. Dies verbessert insbesondere die Unterstützung durch Entwicklungsumgebungen und Typprüfwerkzeuge und hilft dabei, den Code konsistenter zu schreiben. Das Attribut `field(default_factory=list)` stellt sicher, dass jede Instanz ihre eigene Liste erhält und verhindert unbeabsichtigte Seiteneffekte durch gemeinsam genutzte Datenstrukturen.[1]

3 Service Klassen

Service Klassen bündeln fachliche Logik, die mehrere Domain Objekte betrifft. Einfache Berechnungen, die eindeutig zu einem Objekt gehören, dürfen in Domain Klassen verbleiben. Sobald

[1] Eine ausführliche Erläuterung dieses Problems findet sich in Anhang A.

Berechnungen jedoch mehrere Objekte betreffen oder komplexer werden, ist ein Service die bessere Wahl.

Datei: `noten_service.py`

```python
from __future__ import annotations
from modul import Modul

class NotenService:
    """Service-Klasse: enthält fachliche Logik (Berechnungen)."""

    def berechne_gesamt_ects(self, module: list[Modul]) -> int:
        return sum(m.ects for m in module)

    def berechne_gesamtpunkte(self, module: list[Modul]) -> int:
        punkte = 0
        for modul in module:
            for leistung in modul.pruefungsleistungen:
                if leistung.punkte is not None:
                    punkte += leistung.punkte
        return punkte
```

Service Klassen folgen dem Single Responsibility Principle, da sie sich nur bei Änderungen der fachlichen Regeln ändern.

4 Repository Klassen

Die Herkunft von Daten kann sich im Laufe eines Projekts ändern, etwa durch einen Wechsel der Datenspeicherstrategie von Beispieldaten zu Dateien oder Datenbanken. Repository Klassen kapseln diese Änderungen und halten den restlichen Code unabhängig von der konkreten Speicherung.

Datei: `modul_repository.py`

```python
from __future__ import annotations
from modul import Modul

class ModulRepository:
    """
    Repository (Repo): kapselt den Zugriff auf persistente Moduldaten.

    In dieser Phase bewusst als Dummy umgesetzt.
    Später austauschbar durch Datei- oder Datenbankzugriffe.
    """

    def lade_alle(self) -> list[Modul]:
```

```
        return []

    def speichere(self, modul: Modul) -> None:
        pass
```

Dieses Vorgehen folgt dem Dependency Inversion Principle aus den SOLID Regeln.

5 Controller Klassen

Controller Klassen koordinieren den Ablauf der Anwendung. Sie importieren andere Klassen aus separaten Dateien, rufen Services und Repositories auf und bereiten die Ergebnisse für die Darstellung vor. Die expliziten Imports machen Abhängigkeiten sichtbar und unterstützen das Architekturverständnis.

Datei: `dashboard_controller.py`

```python
from __future__ import annotations
from modul_repository import ModulRepository
from noten_service import NotenService

class DashboardController:
    """Controller: vermittelt zwischen UI (View) und fachlicher Logik/Daten."""

    def __init__(self, noten_service: NotenService, modul_repository:
    ↪ ModulRepository):
        self.noten_service = noten_service
        self.modul_repository = modul_repository

    def lade_dashboard_daten(self) -> dict:
        module = self.modul_repository.lade_alle()
        return {
            "module": module,
            "gesamt_ects": self.noten_service.berechne_gesamt_ects(module),
            "gesamtpunkte": self.noten_service.berechne_gesamtpunkte(module),
        }
```

6 View Klassen

View Klassen sind ausschließlich für die Darstellung zuständig und enthalten keine fachliche Logik.

Datei: `leistungsuebersicht_view.py`

```python
class LeistungsUebersichtView:
    """View: zuständig für die Darstellung, keine Berechnungslogik."""
```

```
3
4       def zeige(self, daten: dict) -> None:
5           print(f"ECTS gesamt: {daten['gesamt_ects']}")
6           print(f"Gesamtpunkte: {daten['gesamtpunkte']}")
7           print("Module:")
8           for modul in daten["module"]:
9               print(f"- {modul.name} ({modul.ects} ECTS)")
```

7 Application Klasse

Die Application Klasse ist der Startpunkt des Programms. Sie erzeugt alle Objekte und verbindet diese miteinander. Dieser Schritt wird umgangssprachlich als Verdrahtung bezeichnet. Der fachlich präzisere Begriff ist „Composition Root“.

Datei: dashboard_app.py

```python
from dashboard_controller import DashboardController
from leistungsuebersicht_view import LeistungsUebersichtView
from modul_repository import ModulRepository
from noten_service import NotenService

class DashboardApp:
    """Application (App): Startpunkt und "Verdrahtung" der Schichten."""

    def start(self) -> None:
        repo = ModulRepository()
        service = NotenService()
        controller = DashboardController(service, repo)
        view = LeistungsUebersichtView()

        daten = controller.lade_dashboard_daten()
        view.zeige(daten)

if __name__ == "__main__":
    DashboardApp().start()
```

8 UML der Gesamtarchitektur

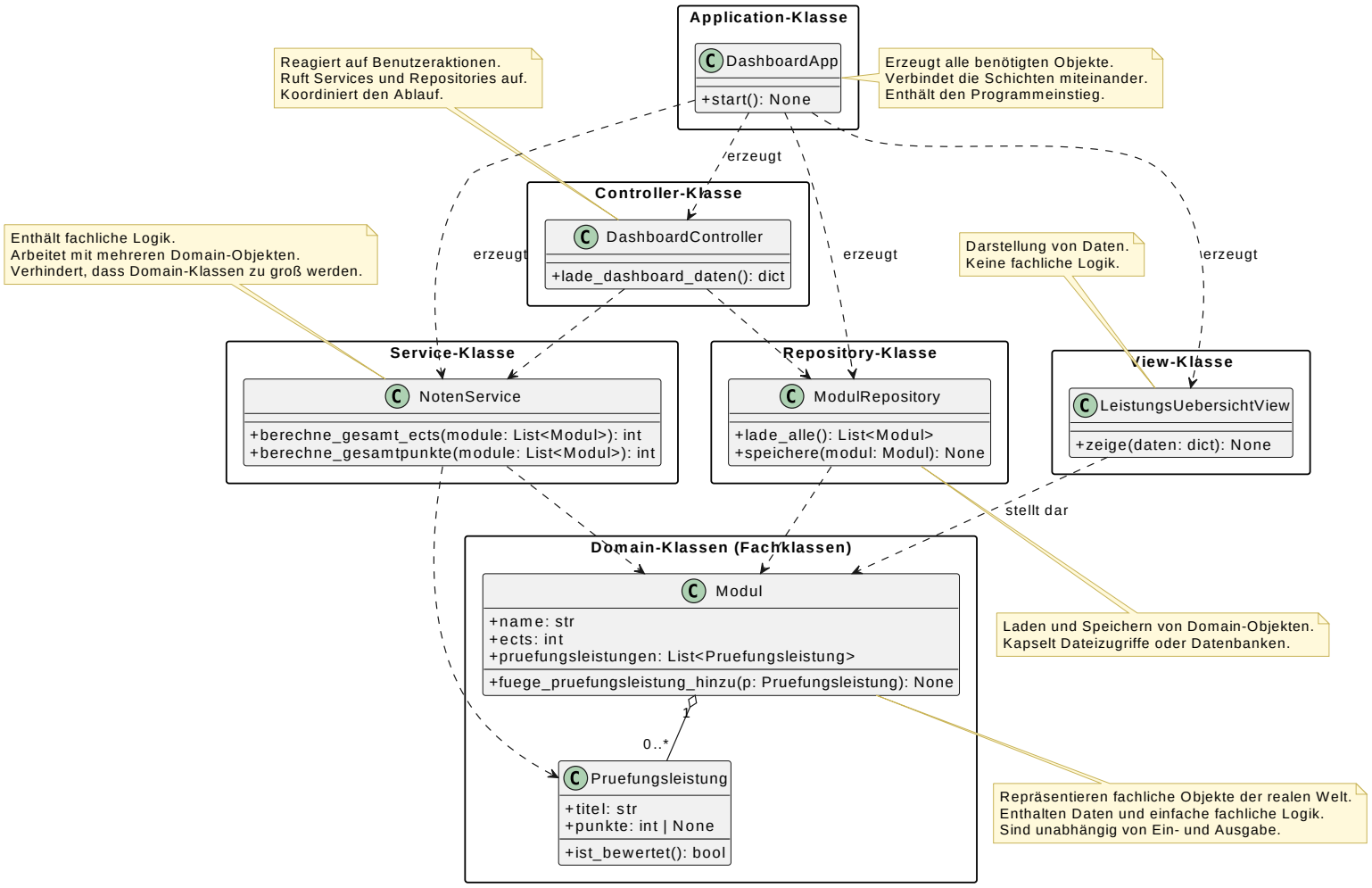


Abbildung 1: Schichtenmodell der Anwendung in Phase 2

Abbildung 1 zeigt die Gesamtarchitektur der Anwendung in Phase 2. Die Application-Klasse bildet den Startpunkt der Anwendung. Die mit *erzeugt* gekennzeichneten Pfeile machen sichtbar, dass hier die benötigten Objekte instanziiert und miteinander verbunden werden. Die View ist über die Beziehung *stellt dar* mit den Domain-Klassen verbunden. Dadurch wird deutlich, dass sie fachliche Daten anzeigt, jedoch keine fachliche Logik enthält. Die übrigen Pfeile verdeutlichen die Zusammenarbeit der Klassen untereinander. Sie zeigen, welche Komponenten andere Komponenten nutzen, ohne deren interne Umsetzung zu kennen.

9 Zusammenfassung

Teil 1 beschreibt einen minimalen, aber sauberen Aufbau für ein Dashboard Projekt. Durch die klare Trennung von Domain-, Service-, Repository-, Controller- und View-Klassen sowie einer Application Klasse entstehen eindeutige Zuständigkeiten. Die Architektur folgt grundlegenden SOLID Prinzipien und bildet eine stabile Grundlage für weitere Ausbaustufen.

Teil II

Erweiterung des Schichtenmodells

Teil I hat eine minimale, aber bereits sauber getrennte Architektur eingeführt. Domain Klassen, Services, Repositories, Controller, Views und die Application Klasse besitzen jeweils klar abgegrenzte Verantwortlichkeiten. Diese Struktur bildet eine stabile Grundlage für kleinere Anwendungen und schafft zugleich die Voraussetzung für weitere Erweiterungen. In diesem Teil wird die bestehende Architektur gezielt weiterentwickelt. Der Schwerpunkt liegt darauf, technische Details stärker von fachlicher Logik zu entkoppeln und einzelne Komponenten austauschbar zu gestalten. Dabei geht es nicht darum, möglichst viele zusätzliche Klassen einzuführen. Vielmehr soll gezeigt werden, wie bestehende Abhängigkeiten bewusster gestaltet werden können, damit Änderungen möglichst lokal bleiben. Ein zentrales Ziel besteht darin, konkrete technische Implementierungen austauschen zu können, ohne höherwertige Programmteile verändern zu müssen. Dadurch wird wichtiger, dass Programmteile nur noch über klar definierte Abstraktionen miteinander kommunizieren. Die folgenden Abschnitte zeigen exemplarisch, wie eine Architektur schrittweise robuster, austauschbarer und erweiterbarer gestaltet werden kann, ohne dabei die Verständlichkeit zu verlieren.

10 Repository Abstraktion

In Teil I wurde bereits eine Repository Klasse eingeführt, um den Zugriff auf gespeicherte Daten von der restlichen Anwendung zu trennen. In dieser Ausbaustufe wird diese Idee weitergeführt. Die Repository Klasse wird nun nicht mehr als konkrete Speicherlösung verstanden, sondern als gemeinsame Abstraktion für verschiedene Speicherstrategien.
Der zentrale Gedanke lautet: Der Controller soll nicht wissen müssen, ob die Daten aus einer JSON Datei, einer SQLite Datenbank oder einer anderen Quelle stammen. Er benötigt nur eine verlässliche Schnittstelle, über die er Module laden und speichern kann. Der Controller hängt also nicht von `JsonModulRepository` oder `SQLiteModulRepository` ab, sondern nur von der abstrakten Klasse `ModulRepository`.
Die allgemeine Repository Klasse definiert, welche Methoden ein Repository anbieten muss. Sie legt aber nicht fest, wie die Daten tatsächlich gespeichert werden.

Datei: `modul_repository_abstract.py`

```
1 from __future__ import annotations
2 from abc import ABC, abstractmethod
3 from modul import Modul
4
5
6 class ModulRepository(ABC):
7     """Abstrakte Repository Klasse.
8
```

```
    Diese Klasse definiert, welche Methoden ein Repository anbieten muss.
    Wie die Daten gespeichert werden, wird hier bewusst nicht festgelegt.
    """

    @abstractmethod
    def lade_alle(self) -> list[Modul]:
        """Lädt alle Module aus einer Datenquelle."""
        pass

    @abstractmethod
    def speichere(self, modul: Modul) -> None:
        """Speichert ein einzelnes Modul in einer Datenquelle."""
        pass
```

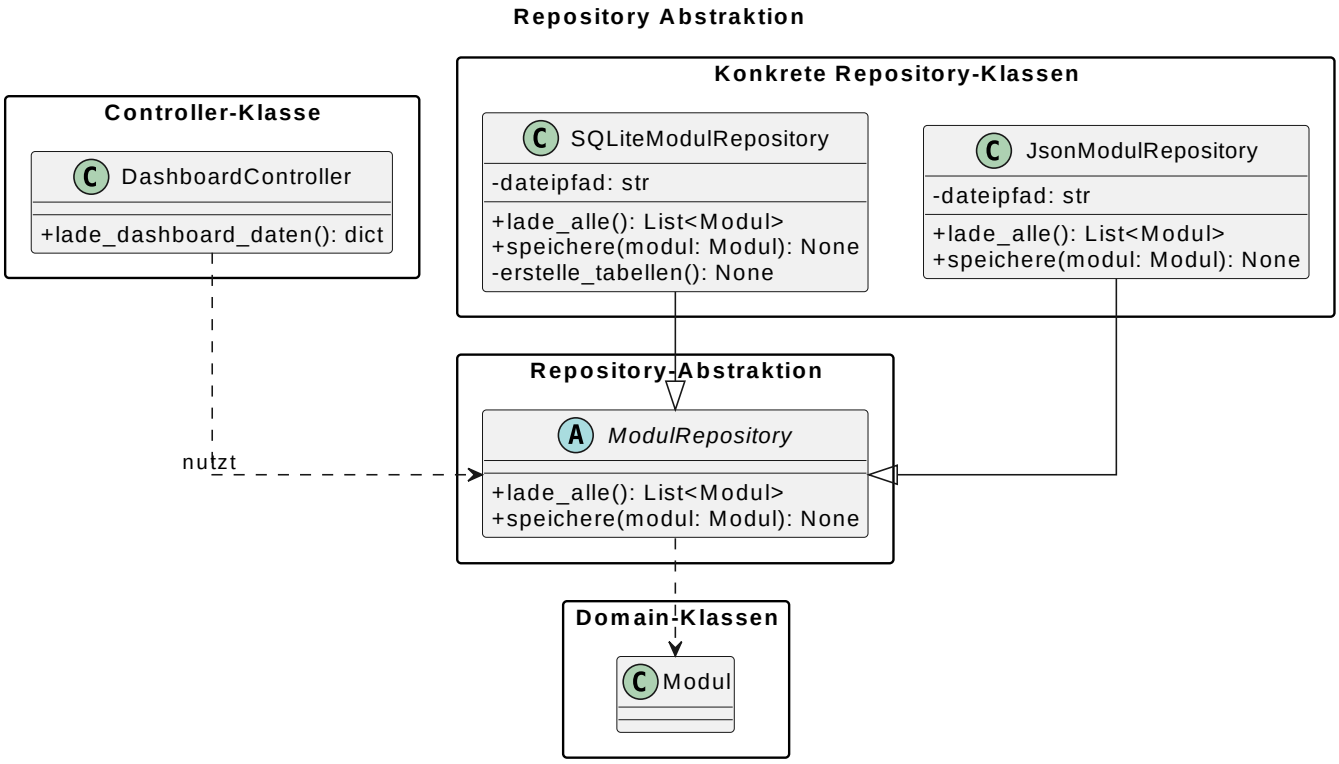


Abbildung 2: Repository Abstraktion mit austauschbaren konkreten Repository Klassen

Abbildung 2 zeigt die grundlegende Struktur der Repository Abstraktion. `ModulRepository` definiert eine gemeinsame Schnittstelle für unterschiedliche Speicherstrategien. Die Klassen `JsonModulRepository` und `SQLiteModulRepository` stellen konkrete Implementierungen dieser Abstraktion dar und bieten dieselben Methoden an.
Der Controller arbeitet ausschließlich mit der allgemeinen Repository Abstraktion. Dadurch muss er nicht wissen, wie die Daten konkret gespeichert werden. Die technische Umsetzung der Speicherung bleibt vollständig in den spezialisierten Repository Klassen gekapselt. Die

konkrete Speicherung wird anschließend in spezialisierten Repository Klassen umgesetzt. Das JSON Repository speichert die Daten in einer Textdatei im JSON Format.

Datei: `json_modul_repository.py`

```python
from __future__ import annotations

import json
from pathlib import Path

from modul import Modul
from modul_repository import ModulRepository
from pruefungsleistung import Pruefungsleistung


class JsonModulRepository(ModulRepository):
    """Konkretes Repository für JSON Dateien."""

    def __init__(self, dateipfad: str):
        self.dateipfad = Path(dateipfad)

    def lade_alle(self) -> list[Modul]:
        if not self.dateipfad.exists():
            return []

        with self.dateipfad.open("r", encoding="utf-8") as datei:
            daten = json.load(datei)

        module: list[Modul] = []
        for modul_daten in daten:
            modul = Modul(
                name=modul_daten["name"],
                ects=modul_daten["ects"],
            )

            for pruefung_daten in modul_daten["pruefungsleistungen"]:
                pruefung = Pruefungsleistung(
                    titel=pruefung_daten["titel"],
                    punkte=pruefung_daten["punkte"],
                )
                modul.fuege_pruefungsleistung_hinzu(pruefung)

            module.append(modul)

        return module

```

```
    def speichere(self, modul: Modul) -> None:
        module = self.lade_alle()
        module.append(modul)

        daten = [self._modul_als_dict(m) for m in module]

        with self.dateipfad.open("w", encoding="utf-8") as datei:
            json.dump(daten, datei, ensure_ascii=False, indent=4)

    def _modul_als_dict(self, modul: Modul) -> dict:
        return {
            "name": modul.name,
            "ects": modul.ects,
            "pruefungsleistungen": [
                {
                    "titel": pruefung.titel,
                    "punkte": pruefung.punkte,
                }
                for pruefung in modul.pruefungsleistungen
            ],
        }
```

Eine alternative Implementierung kann dieselbe Schnittstelle verwenden, aber die Daten in einer SQLite Datenbank speichern.

Datei: `sqlite_modul_repository.py`

```
from __future__ import annotations

import sqlite3
from pathlib import Path

from modul import Modul
from modul_repository import ModulRepository
from pruefungsleistung import Pruefungsleistung


class SQLiteModulRepository(ModulRepository):
    """Konkretes Repository für eine SQLite Datenbank."""

    def __init__(self, dateipfad: str):
        self.dateipfad = Path(dateipfad)
        self._erstelle_tabellen()

    def lade_alle(self) -> list[Modul]:
        with sqlite3.connect(self.dateipfad) as verbindung:
```

```
            cursor = verbindung.cursor()
            cursor.execute("SELECT id, name, ects FROM module")
            modul_zeilen = cursor.fetchall()

            module: list[Modul] = []
            for modul_id, name, ects in modul_zeilen:
                modul = Modul(name=name, ects=ects)

                cursor.execute(
                    """
                    SELECT titel, punkte
                    FROM pruefungsleistungen
                    WHERE modul_id = ?
                    """,
                    (modul_id,),
                )
                pruefungs_zeilen = cursor.fetchall()

                for titel, punkte in pruefungs_zeilen:
                    pruefung = Pruefungsleistung(titel=titel, punkte=punkte)
                    modul.fuege_pruefungsleistung_hinzu(pruefung)

                module.append(modul)

        return module

    def speichere(self, modul: Modul) -> None:
        with sqlite3.connect(self.dateipfad) as verbindung:
            cursor = verbindung.cursor()

            cursor.execute(
                "INSERT INTO module (name, ects) VALUES (?, ?)",
                (modul.name, modul.ects),
            )
            modul_id = cursor.lastrowid

            for pruefung in modul.pruefungsleistungen:
                cursor.execute(
                    """
                    INSERT INTO pruefungsleistungen (modul_id, titel, punkte)
                    VALUES (?, ?, ?)
                    """,
                    (modul_id, pruefung.titel, pruefung.punkte),
                )

```

```
            verbindung.commit()

    def _erstelle_tabellen(self) -> None:
        with sqlite3.connect(self.dateipfad) as verbindung:
            cursor = verbindung.cursor()

            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS module (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    ects INTEGER NOT NULL
                )
                """
            )

            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS pruefungsleistungen (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    modul_id INTEGER NOT NULL,
                    titel TEXT NOT NULL,
                    punkte INTEGER,
                    FOREIGN KEY (modul_id) REFERENCES module(id)
                )
                """
            )

            verbindung.commit()
```

Der Controller bleibt dabei unverändert. Er verwendet nur die allgemeine Repository Abstraktion. Dadurch kann die Speicherstrategie ausgetauscht werden, ohne dass der Controller angepasst werden muss.

Datei: `dashboard_controller_abstract.py`

```
from __future__ import annotations

from modul_repository import ModulRepository
from noten_service import NotenService


class DashboardController:
    """Controller, der nur die abstrakte Repository Klasse kennt."""

```

```
    def __init__(self, noten_service: NotenService, modul_repository:
    ↪ ModulRepository):
        self.noten_service = noten_service
        self.modul_repository = modul_repository

    def lade_dashboard_daten(self) -> dict:
        module = self.modul_repository.lade_alle()
        return {
            "module": module,
            "gesamt_ects": self.noten_service.berechne_gesamt_ects(module),
            "gesamtpunkte": self.noten_service.berechne_gesamtpunkte(module),
        }
```

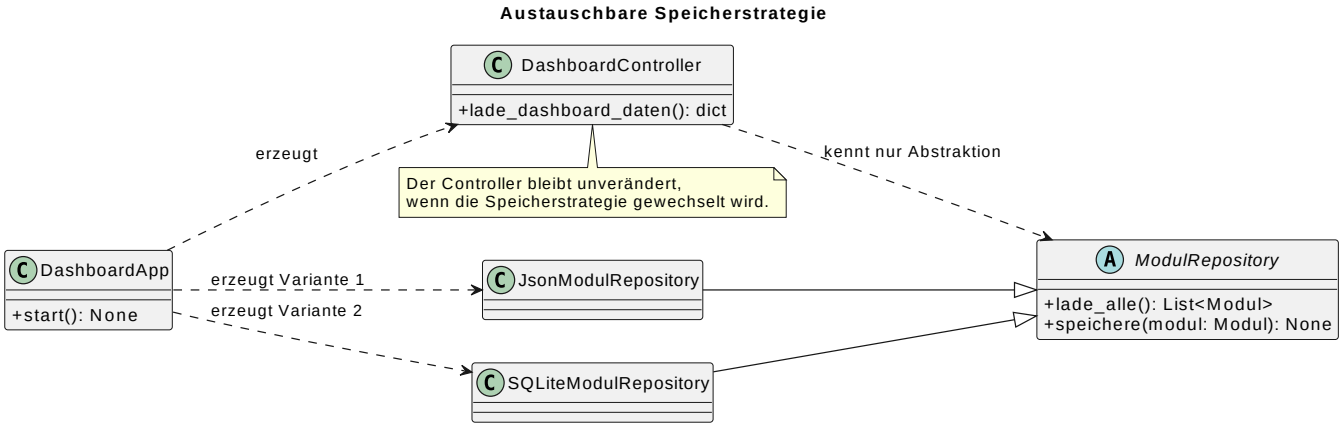


Abbildung 3: Austausch der Speicherstrategie ohne Änderung des Controllers

Abbildung 3 verdeutlicht den praktischen Nutzen der Repository Abstraktion. Die Application Klasse entscheidet beim Start der Anwendung, welche konkrete Repository Implementierung erzeugt wird. Der Controller kennt dabei ausschließlich die allgemeine Klasse `ModulRepository`. Ob intern ein JSON Repository oder ein SQLite Repository verwendet wird, bleibt für den Controller unsichtbar. Dadurch kann die Speicherstrategie ausgetauscht werden, ohne den Controller verändern zu müssen. Genau dieses Vorgehen entspricht dem *Dependency Inversion Principle* (DIP) der SOLID Regeln: Höherwertige Programmteile hängen nicht von konkreten technischen Details ab, sondern nur von allgemeinen Abstraktionen. Die Entscheidung, welche konkrete Repository Klasse verwendet wird, liegt in der Application Klasse. Dort wird das passende Repository erzeugt und an den Controller übergeben.

Datei: `dashboard_app_json.py`

```
from dashboard_controller import DashboardController
from json_modul_repository import JsonModulRepository
from leistungsuebersicht_view import LeistungsUebersichtView
from noten_service import NotenService
```

```
5
6
7  class DashboardAppJson:
8      """Application Klasse mit JSON Repository."""
9
10     def start(self) -> None:
11         repository = JsonModulRepository("module.json")
12         service = NotenService()
13         controller = DashboardController(service, repository)
14         view = LeistungsUebersichtView()
15
16         daten = controller.lade_dashboard_daten()
17         view.zeige(daten)
18
19
20 if __name__ == "__main__":
21     DashboardAppJson().start()
```

Datei: dashboard_app_sqlite.py

```
1  from dashboard_controller import DashboardController
2  from leistungsuebersicht_view import LeistungsUebersichtView
3  from noten_service import NotenService
4  from sqlite_modul_repository import SQLiteModulRepository
5
6
7  class DashboardAppSQLite:
8      """Application Klasse mit SQLite Repository."""
9
10     def start(self) -> None:
11         repository = SQLiteModulRepository("module.db")
12         service = NotenService()
13         controller = DashboardController(service, repository)
14         view = LeistungsUebersichtView()
15
16         daten = controller.lade_dashboard_daten()
17         view.zeige(daten)
18
19
20 if __name__ == "__main__":
21     DashboardAppSQLite().start()
```

Diese Struktur zeigt den praktischen Nutzen einer sauberen Schichtenarchitektur. Die konkrete Speichertechnik ist austauschbar, während Controller, Service, View und Domain Klassen stabil bleiben.

11 Strukturierte Datenübergabe mit DTOs

In den bisherigen Beispielen liefert der Controller die Dashboard Daten als allgemeines `dict` zurück. Diese Lösung ist für erste Beispiele bewusst einfach gehalten, besitzt jedoch einige Nachteile. Es ist nicht direkt erkennbar, welche Schlüssel das Dictionary enthält oder welche Datentypen erwartet werden. Tippfehler in Schlüsselnamen werden häufig erst zur Laufzeit sichtbar. Außerdem bleibt unklar, welche Daten zwischen Controller und View tatsächlich übertragen werden sollen. Um die Datenübergabe zwischen Controller und View klarer und robuster zu gestalten, können sogenannte DTOs verwendet werden. DTO steht für *Data Transfer Object*. Ein DTO ist eine einfache Klasse, deren Aufgabe ausschließlich darin besteht, Daten strukturiert zwischen Schichten zu übertragen. Ein DTO enthält in der Regel keine oder nur sehr wenig fachliche Logik. Es dient vor allem dazu, die erwartete Datenstruktur explizit zu beschreiben.

DTO Klasse

Im folgenden Beispiel wird ein DTO eingeführt, das alle Daten enthält, die für die Darstellung des Dashboards benötigt werden.

Datei: `dashboard_daten_dto.py`

```python
from __future__ import annotations
from dataclasses import dataclass
from modul import Modul


@dataclass
class DashboardDatenDTO:
    """DTO für die Übergabe vorbereiteter Dashboard-Daten."""

    module: list[Modul]
    gesamt_ects: int
    gesamtpunkte: int
```

Die Klasse beschreibt explizit, welche Informationen zwischen Controller und View übertragen werden. Statt allgemeiner Schlüssel-Wert Paare werden klar benannte Attribute verwendet.

Anpassung des Controllers

Der Controller liefert nun kein allgemeines Dictionary mehr zurück, sondern ein klar typisiertes DTO Objekt.

Datei: `dashboard_controller_dto.py`

```python
from __future__ import annotations

from dashboard_daten_dto import DashboardDatenDTO
```

```python
from modul_repository import ModulRepository
from noten_service import NotenService


class DashboardController:
    """Controller mit strukturierter Datenübergabe über ein DTO."""

    def __init__(
        self,
        noten_service: NotenService,
        modul_repository: ModulRepository,
    ):
        self.noten_service = noten_service
        self.modul_repository = modul_repository

    def lade_dashboard_daten(self) -> DashboardDatenDTO:
        module = self.modul_repository.lade_alle()

        return DashboardDatenDTO(
            module=module,
            gesamt_ects=self.noten_service.berechne_gesamt_ects(module),
            gesamtpunkte=self.noten_service.berechne_gesamtpunkte(module),
        )
```

Die Methode `lade_dashboard_daten()` erzeugt ein Objekt der Klasse `DashboardDatenDTO`. Dieses Objekt enthält bereits alle vorbereiteten Informationen, die später von der View benötigt werden. Dadurch wird die Schnittstelle zwischen Controller und View expliziter und leichter nachvollziehbar.

Anpassung der View

Auch die View arbeitet jetzt mit dem DTO. Dadurch wird unmittelbar sichtbar, welche Daten für die Darstellung verfügbar sind.

Datei: `leistungsuebersicht_view_dto.py`

```python
from dashboard_daten_dto import DashboardDatenDTO


class LeistungsUebersichtView:
    """View mit strukturierter Datenübergabe über ein DTO."""

    def zeige(self, daten: DashboardDatenDTO) -> None:
        print(f"ECTS gesamt: {daten.gesamt_ects}")
        print(f"Gesamtpunkte: {daten.gesamtpunkte}")

```

```
        print("Module:")

        for modul in daten.module:
            print(f"- {modul.name} ({modul.ects} ECTS)")
```

Die View greift nun direkt auf die Attribute des DTOs zu. Entwicklungsumgebungen können dadurch die vorhandenen Attribute besser erkennen und unterstützen. Gleichzeitig reduziert sich die Gefahr von Tippfehlern, da nicht mehr mit frei gewählten Dictionary Schlüsseln gearbeitet wird.

Verwendung in der Application Klasse

Die DTO Struktur lässt sich direkt in die bestehende Application Klasse integrieren. Dabei werden lediglich die bisherigen Controller- und View-Klassen durch die DTO Varianten ersetzt.

Datei: `dashboard_app_dto.py`

```python
from dashboard_controller_dto import DashboardController
from leistungsuebersicht_view_dto import LeistungsUebersichtView
from json_modul_repository import JsonModulRepository
from noten_service import NotenService


class DashboardAppDTO:
    """Application (App): Startpunkt mit strukturierter Datenübergabe über
    ↪ DTO."""

    def start(self) -> None:
        repo = JsonModulRepository("module.json")
        service = NotenService()
        controller = DashboardController(service, repo)
        view = LeistungsUebersichtView()

        daten = controller.lade_dashboard_daten()
        view.zeige(daten)


if __name__ == "__main__":
    DashboardAppDTO().start()
```

Die grundlegende Schichtenarchitektur bleibt dabei unverändert. Neu ist lediglich, dass Controller und View nun über ein klar strukturiertes DTO Objekt kommunizieren.

UML der DTO Struktur

Abbildung 4 zeigt die strukturierte Datenübergabe zwischen Controller und View. Beide Klassen arbeiten nun mit der gemeinsamen Klasse `DashboardDatenDTO` statt mit einer allgemeinen Dictionary Struktur. Dadurch werden die Schnittstellen zwischen den Schichten expliziter und klarer nachvollziehbar.

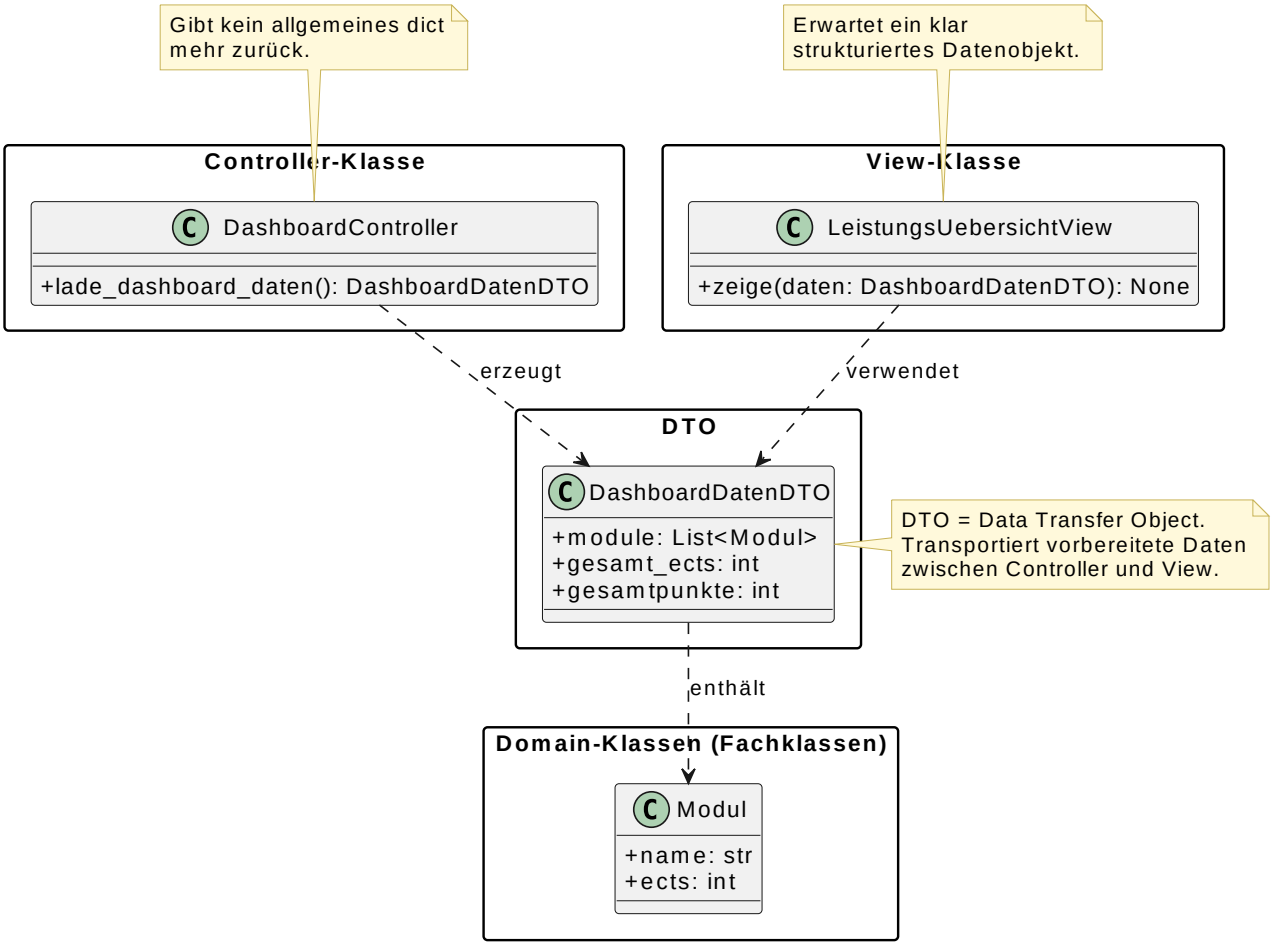


Abbildung 4: Strukturierte Datenübergabe zwischen Controller und View mithilfe eines DTOs

Teil III

Anhang

A Veränderliche Standardwerte und `default_factory`

In Python werden Standardwerte von Funktionsparametern nicht bei jedem Funktionsaufruf, sondern bereits beim Definieren der Funktion erzeugt. Dieses Verhalten betrifft alle Standardparameter, ist aber besonders problematisch bei veränderlichen Objekten wie Listen oder Dictionaries.

Das folgende Beispiel zeigt eine Klasse ohne Verwendung von `@dataclass`. Hier wird eine leere Liste direkt als Standardwert verwendet. Obwohl der Code auf den ersten Blick korrekt wirkt, führt er zu unerwartetem Verhalten.

Datei: `modul_fehlerhaft.py`

```python
class Modul:
    def __init__(self, name, pruefungen=[]):
        self.name = name
        self.pruefungen = pruefungen

    def fuege_pruefung_hinzu(self, pruefung):
        self.pruefungen.append(pruefung)
```

Der entscheidende Punkt ist, dass die leere Liste nur ein einziges Mal erzeugt wird. Alle Instanzen der Klasse teilen sich dieselbe Liste. Wird die Liste bei einem Objekt verändert, ist diese Änderung auch bei allen anderen Objekten sichtbar. Dieser Effekt tritt oft erst später auf und ist daher schwer zu debuggen.
Eine klassische und korrekte Lösung ohne `@dataclass` besteht darin, den Standardwert auf `None` zu setzen und im Konstruktor explizit eine neue Liste zu erzeugen.

Datei: `modul_korrekt_ohne_dataclass.py`

```python
class Modul:
    def __init__(self, name, pruefungen=None):
        self.name = name
        if pruefungen is None:
            self.pruefungen = []
        else:
            self.pruefungen = pruefungen

    def fuege_pruefung_hinzu(self, pruefung):
        self.pruefungen.append(pruefung)
```

In diesem Fall wird bei jeder neuen Instanz eine eigene Liste erzeugt. Der Nachteil dieser Lösung ist der zusätzliche technische Code, der notwendig ist, um das Problem korrekt zu behandeln. Verwendet man `@dataclass`, kann dieses Muster deutlich kompakter formuliert werden. Die Funktion `field(default_factory=list)` gibt an, dass bei jeder neuen Instanz eine neue leere Liste erzeugt werden soll.

Datei: `modul_dataclass_default_factory.py`

```python
from dataclasses import dataclass, field

@dataclass
class Modul:
    name: str
```

```
6       pruefungen: list[str] = field(default_factory=list)
7
8       def fuege_pruefung_hinzu(self, pruefung: str) -> None:
9           self.pruefungen.append(pruefung)
```

Der Begriff *default_factory* ist dabei wörtlich zu verstehen. Es wird kein fester Standardwert angegeben, sondern eine Funktion, die den Standardwert bei Bedarf neu erzeugt. Dadurch erhält jedes Objekt seine eigene Datenstruktur, und unbeabsichtigte Seiteneffekte zwischen verschiedenen Instanzen werden zuverlässig vermieden.
`field(default_factory=list)` ist keine optionale Komfortfunktion, sondern ein wichtiges Werkzeug, um eine häufige und schwer erkennbare Fehlerquelle in Python zu vermeiden.